LAPORAN PRAKTIKUM STRUKTUR DATA

PEKAN 1 TENTANG ADT

Disusun oleh :

M. Fajar Fadhilul Zikri

NIM:**2511533023**

Dosen Pengampu: **Dr. Wahyudi, S.T, M.T.**

Asisten Praktikum: **SHERLY SUKMADIRA PUTRI**

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

TAHUN 2026

DAFTAR ISI

KATA PENGANTAR……. I

BAB 1 PENDAHULUAN…….1

1.1 Pengertian Pratikum

1.2 Tujuan Pratikum

1.3 Persyaratan Pratikum

1.4 Waktu dan Tempat Pratikum

1.5 Manfaat

BAB 2 ISI

2.1 Dasar Teori

2.2 gambar program ADT JAM

BAB 3 PENUTUP

3.1 Kesimpulan

3.2 Saran

Daftar pustaka

Kata pengantar

Puji syukur penulis panjatkan kepada Tuhan Yang Maha Esa karena berkat rahmat dan karunia-Nya laporan praktikum ini dapat diselesaikan dengan baik. Laporan ini disusun untuk memenuhi tugas praktikum mata kuliah/bidang studi Pemrograman Java, dengan fokus pembahasan mengenai tipe data dan variabel.

Penulis menyadari bahwa laporan ini masih jauh dari sempurna, baik dari segi isi maupun penyajian. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi perbaikan di masa mendatang. Semoga laporan ini dapat memberikan manfaat dan menambah pengetahuan bagi pembaca.

Bab I
PENDAHULUAN

1.1 Pengertian Praktikum

Praktikum Struktur Data adalah kegiatan pembelajaran yang dilakukan di laboratorium komputer untuk mengasah keterampilan mahasiswa dalam memahami serta menerapkan konsep struktur data. Kegiatan ini tidak hanya menekankan pada penguasaan teori, tetapi juga pada latihan implementasi struktur data, pengujian, hingga analisis hasil eksekusi. Praktikum dipandang sebagai wahana latihan yang menjembatani pemahaman konseptual dengan kemampuan teknis dalam pengolahan data dan algoritma.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum antara lain sebagai berikut:

1. Membantu mahasiswa memahami konsep dasar struktur data melalui penerapan langsung.
2. Melatih kemampuan mengimplementasikan berbagai struktur data serta menguji program yang dibuat.
3. Meningkatkan keterampilan dalam memecahkan masalah (problem solving) dengan pendekatan algoritmik.
4. Membiasakan mahasiswa bekerja secara sistematis dalam menyusun laporan yang memuat analisis hasil praktikum.
5. Menanamkan sikap teliti, disiplin, serta tanggung jawab dalam melaksanakan kegiatan laboratorium.

1.3 Persyaratan Praktikum

Agar praktikum berjalan lancar, mahasiswa perlu memenuhi beberapa persyaratan berikut:

1. Telah mengikuti perkuliahan teori Struktur Data sebagai dasar pemahaman.
2. Membawa perlengkapan yang diperlukan, antara lain laptop atau komputer yang telah terpasang perangkat lunak pemrograman yang digunakan (misalnya compiler atau IDE).
3. Mengikuti setiap sesi praktikum sesuai jadwal yang ditetapkan dan hadir minimal sesuai ketentuan program studi.
4. Mematuhi tata tertib laboratorium, termasuk menjaga keamanan data, perangkat, serta lingkungan kerja.
5. Menyusun laporan praktikum dengan format dan aturan yang telah ditetapkan dalam pedoman.

1.4 Waktu dan Tempat Praktikum

Pelaksanaan praktikum Struktur Data mengikuti kalender akademik yang berlaku pada program studi. Setiap sesi praktikum dilaksanakan sesuai jadwal yang ditentukan oleh dosen pengampu. Tempat kegiatan umumnya berlangsung di laboratorium komputer, namun pada kondisi tertentu dapat dilaksanakan secara

mandiri dengan perangkat masing-masing, selama memenuhi syarat teknis yang ditetapkan.

1.5 Manfaat

Manfaat praktikum ini adalah untuk memperkuat pemahaman konsep struktur data, melatih kemampuan problem solving, serta mendorong kreativitas dan inovasi mahasiswa dalam menyelesaikan permasalahan yang berkaitan dengan pengolahan data secara efisien.

BAB II

ISI :

2.1 Dasar Teori

Abstract Data Type (ADT) adalah konsep dalam ilmu komputer yang mendefinisikan tipe data secara abstrak, yaitu melalui operasi-operasi yang dapat dilakukan terhadap data tersebut tanpa memedulikan bagaimana data tersebut diimplementasikan secara internal. ADT memisahkan antara **apa** yang dilakukan sebuah tipe data dengan **bagaimana** cara kerjanya.

ADT berfokus pada definisi **operasi dasar** seperti penambahan data, penghapusan, pencarian, dan pengaksesan, serta sifat-sifat yang harus dipenuhi oleh operasi-operasi tersebut. Contoh ADT yang umum digunakan antara lain:

- **List**: Koleksi data berurutan yang dapat ditambah, dihapus, dan diakses berdasarkan posisi.
- **Stack**: Struktur data dengan prinsip Last In First Out (LIFO), yaitu data terakhir yang masuk akan keluar terlebih dahulu.
- **Queue**: Struktur data dengan prinsip First In First Out (FIFO), yaitu data pertama yang masuk akan keluar terlebih dahulu.
- **Set**: Kumpulan data unik tanpa urutan tertentu.
- **Map (atau Dictionary)**: Kumpulan pasangan key-value yang menghubungkan kunci unik dengan nilai tertentu.

Implementasi ADT bisa berbeda-beda, misalnya menggunakan array, linked list, atau struktur data lainnya. Namun, pengguna ADT hanya perlu memahami operasi yang disediakan dan sifat-sifatnya tanpa harus mengetahui detail implementasi.

Penggunaan ADT mempermudah pengembangan perangkat lunak karena memungkinkan penggantian implementasi tanpa mengubah kode yang menggunakan ADT tersebut, sehingga meningkatkan modularitas dan pemeliharaan program.

2.2 gambar code program ADT Jam class Jam:

```
public class Jam_2511533023 {
    private int hh; // 0..23
    private int mm; // 0..59
    private int ss; // 0..59

    // --- Validator ---
    // Mengecek apakah nilai jam, menit, dan detik valid sesuai batasnya
    public static boolean isValid(int h, int m, int s) {
        return (0 <= h && h <= 23) &&
               (0 <= m && m <= 59) &&
               (0 <= s && s <= 59);
    }
```

```java
    // Konstruktor
    // Membuat objek Jam dengan validasi nilai waktu
    public Jam_2511533023(int h, int m, int s) {
        if (!isValid(h, m, s)) {
            throw new IllegalArgumentException("Waktu tidak valid");
        }
        this.hh = h;
        this.mm = m;
        this.ss = s;
    }

    // Selector
    // Mengambil nilai jam
    public int getHour() { return hh; }
    // Mengambil nilai menit
    public int getMinute() { return mm; }
    // Mengambil nilai detik
    public int getSecond() { return ss; }

    // Mutator
    // Mengubah nilai jam
    public void setHour(int h) { this.hh = h; }
    // Mengubah nilai menit
    public void setMinute(int m) { this.mm = m; }
    // Mengubah nilai detik
    public void setSecond(int s) { this.ss = s; }

    // Konversi
    // Mengubah waktu ke total detik dari tengah malam
    public int toSeconds() {
        return hh * 3600 + mm * 60 + ss;
    }

    // Membuat objek Jam dari total detik (dihitung modulo 24 jam)
    public static Jam_2511533023 fromSecond(int total) {
        if (total < 0) throw new IllegalArgumentException("detik
negatif");

        total %= 24 * 3600; // membungkus nilai detik dalam 24 jam
        int h = total / 3600;
        total %= 3600;
        int m = total / 60;
        int s = total % 60;

        return new Jam_2511533023(h, m, s);
    }

    // Relasional
    // Membandingkan waktu dengan objek Jam lain berdasarkan total detik
    public int compareTo(Jam_2511533023 other) {
        return Integer.compare(this.toSeconds(), other.toSeconds());
    }

    // Mengecek kesamaan waktu dengan objek lain
    public boolean equals(Object o) {
        if (!(o instanceof Jam_2511533023 j)) return false;
        return hh == j.hh && mm == j.mm && ss == j.ss;
```

```java
    }

    // Menghasilkan kode hash berdasarkan jam, menit, dan detik
    public int hashCode() {
        return java.util.Objects.hash(hh, mm, ss);
    }

    // Aritmatika
    // Menambahkan waktu dengan waktu lain dan menghasilkan objek baru
    public Jam_2511533023 plus(Jam_2511533023 other) {
        return fromSecond(this.toSeconds() + other.toSeconds());
    }

    // Mengurangi waktu dengan waktu lain dan menghasilkan objek baru
    public Jam_2511533023 minus(Jam_2511533023 other) {
        return fromSecond(Math.floorMod(
            this.toSeconds() - other.toSeconds(),
            24 * 3600
        ));
    }

    // Mendapatkan waktu 1 detik setelah waktu sekarang
    public Jam_2511533023 nextSecond() {
        return fromSecond(this.toSeconds() + 1);
    }

    // Mendapatkan waktu n detik setelah waktu sekarang (n >= 0)
    public Jam_2511533023 nextSecond(int n) {
        return fromSecond(this.toSeconds() + Math.max(0, n));
    }

    // Mendapatkan waktu 1 detik sebelum waktu sekarang
    public Jam_2511533023 prevSecond() {
        return fromSecond(Math.floorMod(
            this.toSeconds() - 1,
            24 * 3600
        ));
    }

    // Mendapatkan waktu n detik sebelum waktu sekarang (n >= 0)
    public Jam_2511533023 prevNSecond(int n) {
        return fromSecond(Math.floorMod(
            this.toSeconds() - Math.max(0, n),
            24 * 3600
        ));
    }

    // Durasi detik
    // Menghitung selisih detik antara waktu awal dan akhir
    public static int durasiDetik(Jam_2511533023 awal, Jam_2511533023
akhir) {
        return akhir.toSeconds() - awal.toSeconds();
    }

    // Mengubah objek Jam ke format string "hh:mm:ss"
    public String toString() {
        return String.format("%02d:%02d:%02d", hh, mm, ss);
```

```
    }
}
```

Program class diatas berfungsi untuk merepresentasikan dan memanipulasi waktu (jam, menit, detik) dalam bentuk objek dengan berbagai kemampuan, sehingga programmer bisa bekerja dengan waktu secara lebih terstruktur dan aman tanpa harus mengurusi detik secara manual.

Selanjutnya ada class JamDriver:

```
public class JamDriver_2511533023 {

    public static void main(String[] args) {
        Jam_2511533023 a = new Jam_2511533023(23, 59, 50);
        Jam_2511533023 b = new Jam_2511533023(0, 0, 15);

        System.out.println("a           = " + a);
        System.out.println("b           = " + b);
        System.out.println("a+b         = " + a.plus(b));
        System.out.println("next 20s    = " + a.nextSecond(20));
        System.out.println("durasi(a,b) = " +
Jam_2511533023.durasiDetik(a, b));
        System.out.println("a.compareTo(b) = " + a.compareTo(b));
    }
}
```

Fungsi dari class ini adalah sebagai program penguji (driver) untuk class **Jam_2511533023**. Fungsinya adalah untuk membuat objek Jam, menjalankan berbagai operasi yang tersedia di class Jam (seperti penjumlahan waktu, penggeseran detik, perhitungan durasi, dan perbandingan waktu), serta menampilkan hasilnya ke layar. Dengan kata lain, class ini digunakan untuk mendemonstrasikan dan menguji semua fitur utama class **Jam_2511533023** agar programmer dapat memastikan bahwa class Jam bekerja sesuai harapan.

Hasil Output:

```
a           = 23:59:50
b           = 00:00:15
a+b         = 00:00:05
next 20s    = 00:00:10
durasi(a,b) = -86375
a.compareTo(b) = 1
```

Selanjutnya ada class JamDriver2:

```
import java.util.Scanner;

public class JamDriver2_2511533023 {

    public static void main(String[] args) {
        Scanner input = new Scanner(System.in); // Membuat Scanner untuk
input dari user

        System.out.println("=== Program Driver Objek Jam ===");

        // 1. Input Jam Pertama
        System.out.println("\n--- Input Jam 1 ---");
        Jam_2511533023 j1 = buatJamDariInput(input); // Meminta user
memasukkan jam pertama dengan validasi

        // 2. Input Jam Kedua
        System.out.println("\n--- Input Jam 2 ---");
        Jam_2511533023 j2 = buatJamDariInput(input); // Meminta user
memasukkan jam kedua dengan validasi

        // 3. Menampilkan Data
        System.out.println("\n--- Hasil Operasi ---");
        System.out.println("Jam 1 (String)      : " + j1); //
Menampilkan jam 1 dalam format hh:mm:ss
        System.out.println("Jam 2 (String)      : " + j2); //
Menampilkan jam 2 dalam format hh:mm:ss
        System.out.println("Jam 1 dalam detik   : " + j1.toSeconds());
// Menampilkan total detik jam 1
        System.out.println("Jam 2 dalam detik   : " + j2.toSeconds());
// Menampilkan total detik jam 2

        // 4. Operasi Relasional
        int perbandingan = j1.compareTo(j2); // Membandingkan jam 1 dan
jam 2
        if (perbandingan > 0) {
            System.out.println("Status              : Jam 1 lebih lambat
(setelah) Jam 2"); // j1 > j2
        } else if (perbandingan < 0) {
            System.out.println("Status              : Jam 1 lebih awal
(sebelum) Jam 2"); // j1 < j2
        } else {
            System.out.println("Status              : Jam 1 dan Jam 2
sama persis"); // j1 == j2
        }

        // 5. Operasi Aritmatika
        System.out.println("Durasi (J1 ke J2)   : " +
                Jam_2511533023.durasiDetik(j1, j2) + " detik"); //
Menghitung selisih detik antara j1 dan j2

        Jam_2511533023 jNext = j1.nextSecond();
        System.out.println("Jam 1 Detik Berikutnya: " + jNext); //
Menampilkan jam 1 + 1 detik

        Jam_2511533023 jPrev = j1.prevSecond();
```

```
        System.out.println("Jam 1 Detik Sebelumnya: " + jPrev); //
Menampilkan jam 1 - 1 detik

        // 6. Operasi Penjumlahan
        Jam_2511533023 jHasilPlus = j1.plus(j2);
        System.out.println("Hasil J1 + J2       : " + jHasilPlus); //
Menjumlahkan jam 1 dan jam 2

        input.close(); // Menutup Scanner
        System.out.println("\nProgram Selesai.");
    }

    /**
     * Prosedur untuk input jam dengan validasi berulang
     */
    private static Jam_2511533023 buatJamDariInput(Scanner sc) {
        int h, m, s;

        while (true) {
            System.out.print("Masukkan Jam (0-23)   : ");
            h = sc.nextInt(); // Input jam

            System.out.print("Masukkan Menit (0-59) : ");
            m = sc.nextInt(); // Input menit

            System.out.print("Masukkan Detik (0-59) : ");
            s = sc.nextInt(); // Input detik

            if (Jam_2511533023.isValid(h, m, s)) {
                return new Jam_2511533023(h, m, s); // Jika valid, buat
objek Jam dan kembalikan
            } else {
                System.out.println("[Error] Input tidak valid!
Ulangi.\n"); // Jika tidak valid, ulangi input
            }
        }
    }
}
```

berfungsi sebagai program interaktif untuk menguji class **Jam_2511533023**. Fungsinya adalah meminta user memasukkan dua waktu (jam, menit, detik), kemudian menampilkan waktu tersebut dalam format string dan total detik. Selain itu, class ini melakukan operasi relasional untuk membandingkan dua jam, menghitung durasi antar jam, menampilkan detik berikutnya dan sebelumnya dari jam pertama, serta menjumlahkan kedua jam. Intinya, class ini digunakan untuk mendemonstrasikan semua fitur utama class **Jam_2511533023** secara interaktif dan memvalidasi input dari pengguna.

Hasil Output:

```
=== Program Driver Objek Jam ===

--- Input Jam 1 ---
Masukkan Jam (0-23)   : 17
Masukkan Menit (0-59) : 18
Masukkan Detik (0-59) : 15

--- Input Jam 2 ---
Masukkan Jam (0-23)   : 15
Masukkan Menit (0-59) : 16
Masukkan Detik (0-59) : 13

--- Hasil Operasi ---
Jam 1 (String)      : 17:18:15
Jam 2 (String)      : 15:16:13
Jam 1 dalam detik   : 62295
Jam 2 dalam detik   : 54973
Status              : Jam 1 lebih lambat (setelah) Jam 2
Durasi (J1 ke J2)   : -7322 detik
Jam 1 Detik Berikutnya: 17:18:16
Jam 1 Detik Sebelumnya: 17:18:14
Hasil J1 + J2       : 08:34:28

Program Selesai.
```

BAB III
PENUTUP

3.1 KESIMPULAN

Dari praktikum ini, dapat disimpulkan bahwa **Abstract Data Type (ADT)** adalah cara membuat tipe data yang fokus pada apa yang bisa dilakukan data tanpa harus tahu cara menyimpannya di komputer. Dengan ADT, program menjadi lebih rapi, mudah diubah, dan aman.

Class Jam_2511533023 adalah contoh ADT yang menyimpan waktu (jam, menit, detik) dan menyediakan operasi seperti menambahkan atau mengurangi detik, menghitung selisih waktu, dan membandingkan dua jam. Program driver (JamDriver_2511533023 dan JamDriver2_2511533023) membantu kita mencoba semua fitur class Jam dan memastikan semuanya bekerja dengan benar, termasuk memeriksa input dari pengguna.

Secara sederhana, praktikum ini mengajarkan kita bagaimana membuat tipe data sendiri, menggunakan objek, dan menjalankan operasi pada data secara terstruktur sehingga lebih mudah dipahami dan diterapkan dalam pemrograman.

3.2 SARAN

Untuk pemahaman yang lebih baik disarankan agar mahasiswa mencoba berbagai input jam dan eksperimen dengan semua operasi pada class Jam, seperti penjumlahan, pengurangan, dan perbandingan, agar lebih memahami cara kerja class ini. Mahasiswa juga dianjurkan membuat program kecil sendiri untuk latihan, serta selalu memberi komentar singkat pada kode agar mudah dimengerti. Praktikum ini membantu memahami konsep ADT, yaitu fokus pada apa yang bisa dilakukan data tanpa harus tahu cara penyimpanannya.

Daftar Pustaka

https://en.wikipedia.org/wiki/Abstract_data_type?utm_source=chatgpt.com
https://www.britannica.com/technology/abstract-data-type?utm_source=chatgpt.com

LAPORAN TUGAS PEKAN 1 ADT

Tugas Praktikum Pekan 1

- Buatlah sebuah program ADT mobil yang memiliki atribut: nama, tahun, cc, harga, dan merk
- Contoh: Avanza, 2020, 1300, 150000000, Toyota
- Buatlah method ADT mobil tersebut: tambah mobil, hapus mobil, Selektor dan mutator
- Buatlah 2 kelas: 1 kelas ADT mobil dan 1 kelas driver. Setiap kelas lengkapi dengan nim

CLASS ADTMobil:

```
import java.util.ArrayList;

public class ADTMobil_2511533023 {
    // atribut mobil: nama, tahun, kapasitas mesin (cc), harga, merk
    private String nama;
    private int tahun;
    private int cc;
    private int harga;
    private String merk;

    // list untuk menyimpan semua objek mobil
    private ArrayList<ADTMobil_2511533023> daftarMobil = new
ArrayList<>();

    // constructor default (tanpa parameter)
    public ADTMobil_2511533023() {}

    // constructor untuk membuat objek mobil dengan data lengkap
    public ADTMobil_2511533023(String nama, int tahun, int cc, int
harga, String merk) {
        this.nama = nama;
        this.tahun = tahun;
        this.cc = cc;
        this.harga = harga;
        this.merk = merk;
    }

    // method untuk menambahkan mobil ke daftar
    public void tambahMobil(ADTMobil_2511533023 mobil) {
        daftarMobil.add(mobil); // menambahkan objek mobil ke ArrayList
    }

    // method untuk menghapus mobil berdasarkan indeks
    public void hapusMobil(int index) {
        if (index >= 0 && index < daftarMobil.size()) {
            daftarMobil.remove(index); // menghapus mobil dari daftar
jika indeks valid
        }
```

```
    }

    // selector / getter untuk tiap atribut
    public String getNama() { return nama; }
    public int getTahun() { return tahun; }
    public int getCc() { return cc; }
    public int getHarga() { return harga; }
    public String getMerk() { return merk; }

    // mutator / setter untuk tiap atribut
    public void setNama(String nama) { this.nama = nama; }
    public void setTahun(int tahun) { this.tahun = tahun; }
    public void setCc(int cc) { this.cc = cc; }
    public void setHarga(int harga) { this.harga = harga; }
    public void setMerk(String merk) { this.merk = merk; }

    // method menampilkan semua mobil di daftar
    public void tampilkanMobil() {
        if (daftarMobil.isEmpty()) { // cek apakah daftar kosong
            System.out.println("Data mobil kosong");
        } else {
            for (ADTMobil_2511533023 m : daftarMobil) { // tampilkan
tiap mobil
                System.out.println(
                    m.getNama() + ", " +
                    m.getTahun() + ", " +
                    m.getCc() + ", " +
                    m.getHarga() + ", " +
                    m.getMerk()
                );
            }
        }
    }
}
```

Class ini berfungsi sebagai representasi Abstract Data Type (ADT) untuk mobil, yaitu menyimpan data mobil seperti nama, tahun, kapasitas mesin, harga, dan merk, serta menyediakan operasi utama untuk mengelola data mobil. Fungsi utamanya meliputi: menambahkan mobil ke daftar, menghapus mobil dari daftar, mengambil dan mengubah atribut mobil (getter dan setter), serta menampilkan semua data mobil yang tersimpan. Dengan class ini, programmer bisa mengelola kumpulan mobil secara terstruktur dan fleksibel tanpa harus memikirkan detail penyimpanan data di ArrayList.

Selanjutnya ada class Driver:

```
public class Driver_2511533023 {
    public static void main(String[] args) {

        ADTMobil_2511533023 dataMobil = new ADTMobil_2511533023();
        // membuat objek ADTMobil untuk menampung daftar mobil
```

```
        // contoh pembuatan objek mobil sesuai soal
        ADTMobil_2511533023 m1 = new ADTMobil_2511533023(
            "Avanza", 2020, 1300, 150000000, "Toyota"
        );
        // membuat objek mobil dengan atribut nama, tahun, cc, harga,
dan merk

        // tambah mobil ke daftar
        dataMobil.tambahMobil(m1);
        // menambahkan mobil m1 ke daftarMobil di objek dataMobil

        // tampilkan daftar mobil sebelum dihapus
        System.out.println("Data Mobil:");
        dataMobil.tampilkanMobil();
        // menampilkan semua mobil yang ada di daftarMobil

        // hapus mobil dari daftar
        dataMobil.hapusMobil(0);
        // menghapus mobil pada indeks 0 (m1) dari daftarMobil

        // tampilkan daftar mobil setelah dihapus
        System.out.println("\nSetelah dihapus:");
        dataMobil.tampilkanMobil();
        // menampilkan daftarMobil setelah m1 dihapus (sekarang kosong)
    }
}
```

Class ini memilik fungsi sebagai program penguji (driver) untuk class ADTMobil_2511533023. Fungsinya adalah untuk membuat objek mobil, menambahkan mobil ke daftar, menampilkan daftar mobil, dan menghapus mobil dari daftar, sehingga dapat mendemonstrasikan dan menguji semua operasi utama pada class ADT Mobil seperti tambah, hapus, dan tampilkan data. Dengan class ini, programmer dapat memastikan bahwa class ADTMobil_2511533023 bekerja sesuai harapan.

Hasil Output:

```
Data Mobil:
Avanza, 2020, 1300, 150000000, Toyota

Setelah dihapus:
Data mobil kosong
```